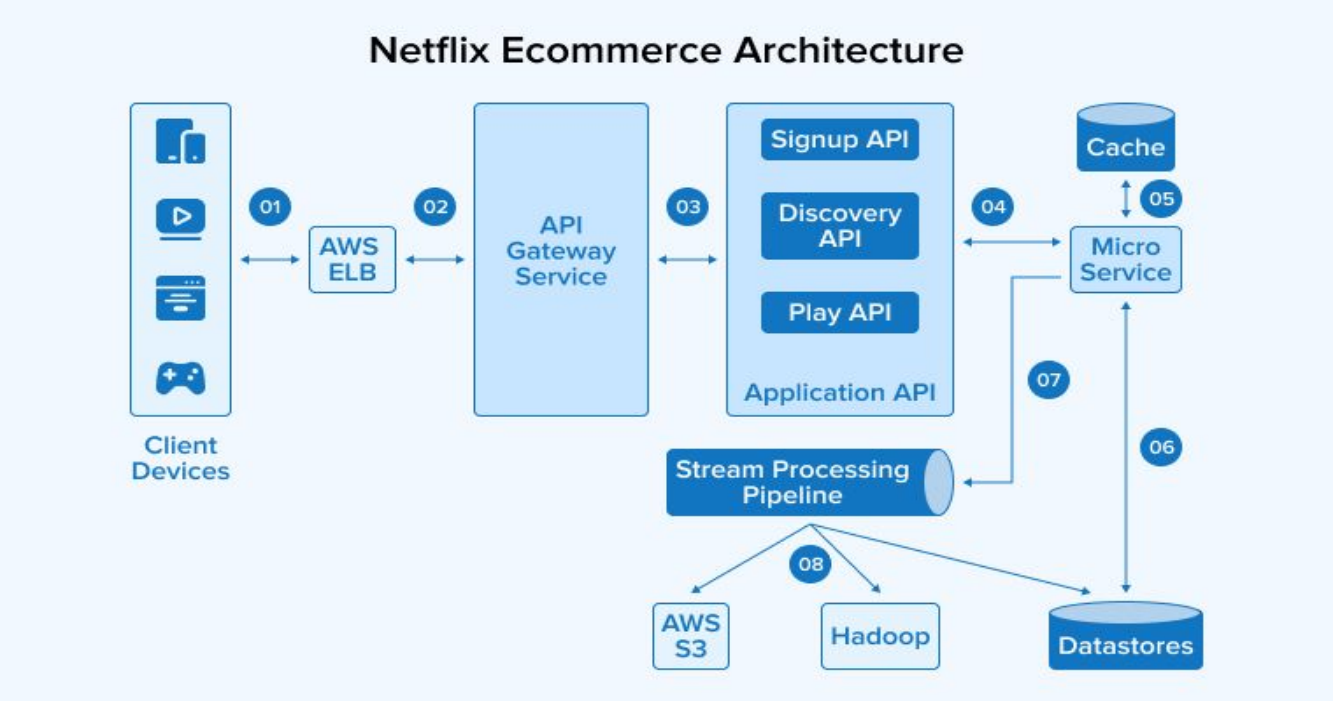
Resilience at Scale in Distributed systems

Sandeep Joshi (Uber)

Microservices in action



https://www.tatvasoft.com/outsourcing/2024/02/e-commerce-microservices-architecture.html

Agenda

1. Retrying Transient failures in network calls
2. Blast Radius & Cellular architecture
3. Randomization
4. Caching

Retrying Transient failures

retrying a network call

At high load, transient failures do happen

1. network connection can timeout and get closed
2. server can temporarily run out of resources

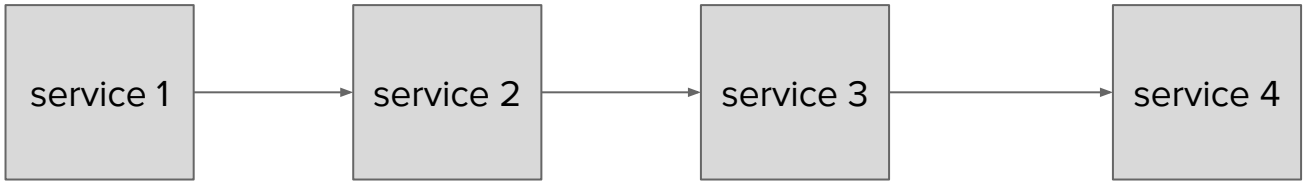
```
for i = 1 to 3
    server.doWork()
```

Dumb retry can hurt !

why is this wrong ?

think...

```
for i = 1 to 3
   err = server.doWork()
   if no err
      break
```



Dumb retry can hurt !

Wasted work (duplicate requests)

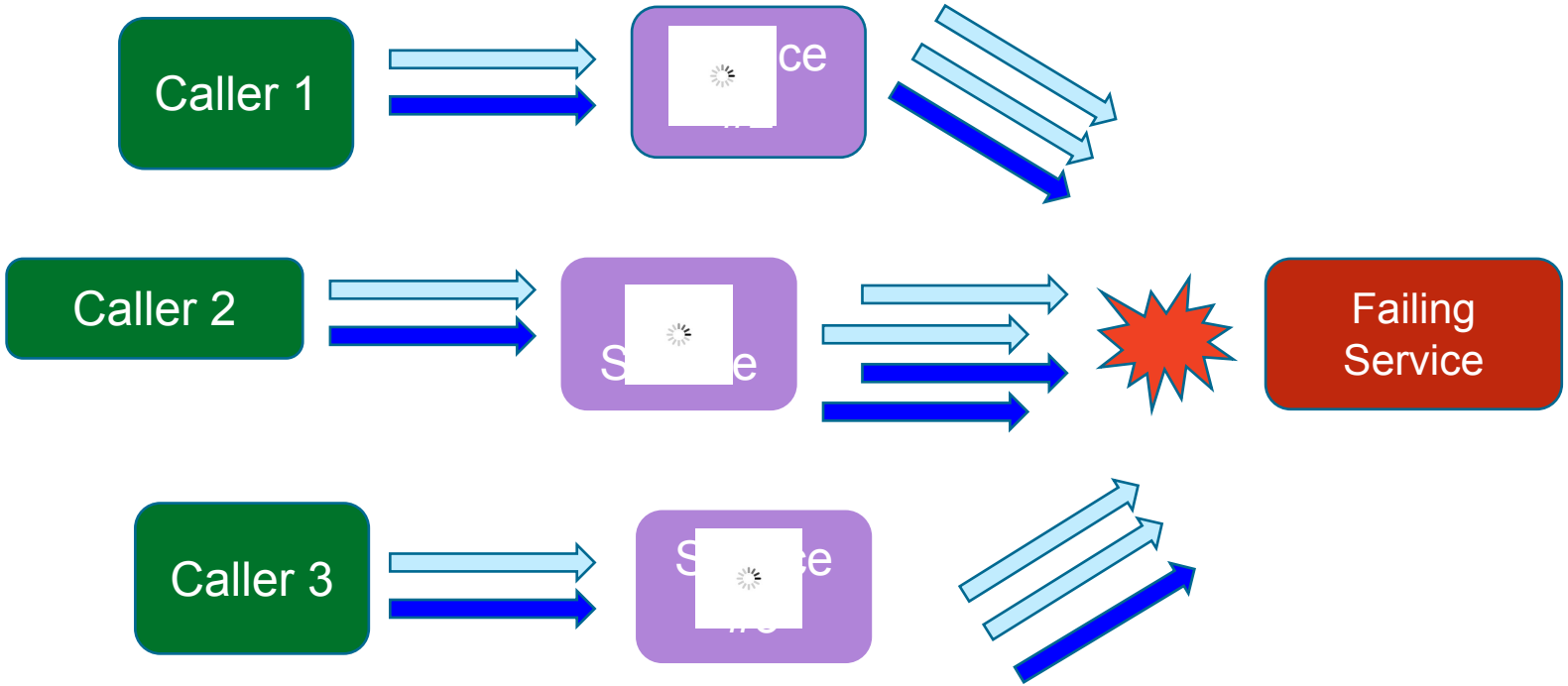
it can exhaust connections on the server

it can exhaust thread pools on the server

It can increase latency on the cascading callers (S1 -> S2 -> S3)

```
for i = 1 to 3
   server.doWork()
```

Brownout (retry storm)



an example of a retry storm

> This happened in part because apps won't accept an error for an answer and start retrying, sometimes aggressively, and in part because end-users also won't take an error for an answer and start reloading the pages, or killing and relaunching their apps, sometimes also aggressively.

Understanding How Facebook Disappeared from the Internet

https://blog.cloudflare.com/october-2021-facebook-outage/

When to retry

Do it only on transient failures (i.e. No point retrying "*invalid*" requests)

Server response must indicate if an error is retryable

Server can specify time after which to retry (e.g. after 5 seconds)

Safe retry requires idempotent APIs

Retry should be harmless if request has already succeeded

This reduces client-side complexity

(e.g. do not return error if request is already done)

(Making retries safe with idempotent APIs)

https://d1.awsstatic.com/builderslibrary/pdfs/making-retries-safe-with-idempotent-apis-malcolm-featonby.pdf

Safe retry requires idempotent APIs

what if it is a "*Create New User*" request ??

should server create another new user with same profile ?

what should the second request return ?

how can server tell if the user was already created ?

Who decides if the request is same ?

POLL

Client should decide	
Server should decide	

Who decides if the request is same ?

Options:

1. Server can use hash(*request*)
2. Client can generate a unique identifier

Situations to consider

1. Client may send twice, but server may receive request only once
2. Client may not see response from server to first request

So its better to have client-generated identifier

Client decides if request is duplicate

Use client token to enable server to identify duplicate requests

ClientToken

Unique, case-sensitive identifier that you provide to ensure the idempotency of the request. For more information, see Ensuring Idempotency.

Type: String

Required: No

https://docs.aws.amazon.com/AWSEC2/latest/APIReference/API_AllocateHosts.html

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-direct-api-idempotency.html

Avoid wasted work

Indicate the expiration time on every request

If server is slow, it can throw away expired requests

After a restart, server can process newer requests before older

Exponential growth in retries

Retries can grow exponentially as call stack increases

(S1 -> S2 -> S3 -> S4) : S1 sees failure of S4; every retry from S1 also reaches S4

service	retries	total
S1	3	3
S2	3 x 3	9
S3	9 x 3	27
S4	27 x 3	81

When to stop retries

Retries can grow exponentially as call stack increases

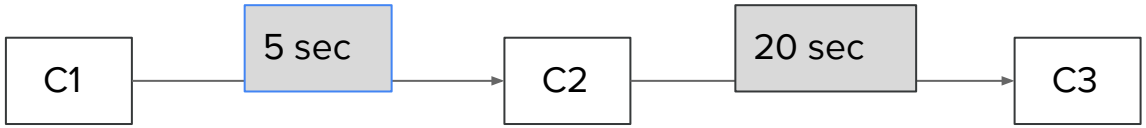
(S1 -> S2 -> S3 -> S4) : S1 sees failure of S4; every retry from S1 also reaches S4

Use a Retry budget to avoid "call amplification" (i.e. linear growth in retries)

ClientConfiguration.withThrottledRetries()

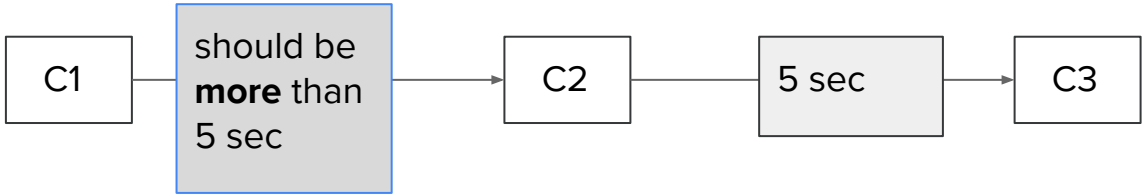
https://aws.amazon.com/blogs/developer/introducing-retry-throttling/

what is the problem ?



timeout settings in calls

Finish before your caller times out



ClientConfiguration.withClientExecutionTimeout()

Sets the amount of time (in milliseconds) to allow the client to complete the execution of an API call. This timeout covers the entire client execution except for marshalling. This includes request handler execution, all HTTP request including ***retries****, unmarshalling, etc.*

https://docs.aws.amazon.com/AWSJavaSDK/latest/javadoc/com/amazonaws/ClientConfiguration.html#withClientExecutionTimeout-int-

best practices : Server

1. Build Idempotent APIs
2. Expose retry info in response (e.g. is retryable, retry-interval)
3. Throw away expired requests
4. Enforce limits (pending jobs, TPS) in order to estimate the retry interval

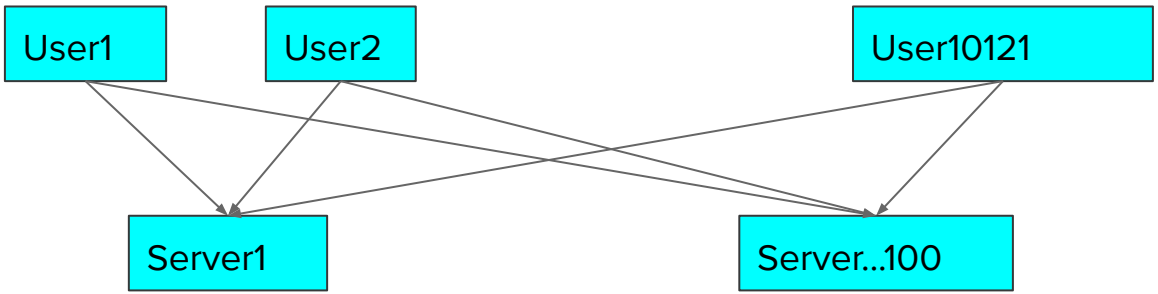
best practices : Client

1. Use a token to indicate duplicate request
2. Only retry if Server says its a retryable failure
3. Only retry after time given by Server
4. Throttle the retries if too many requests are failing (use "budget")

Blast Radius & Cellular architecture

Whats wrong with this config

Any user can be served by any backend server

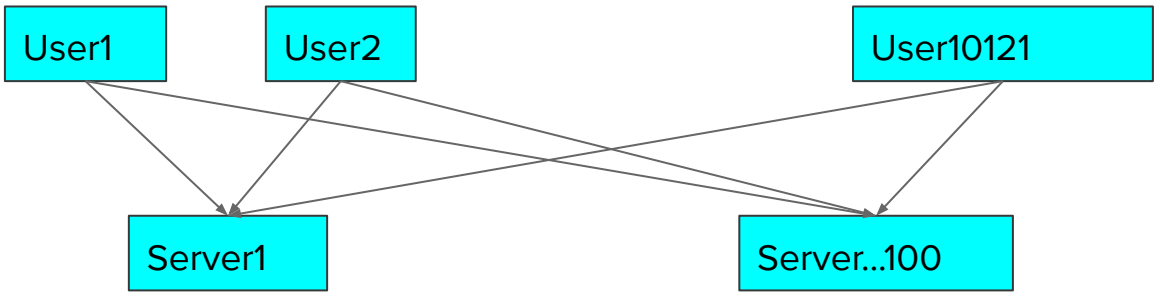


Blast radius : Perils of N x N

N x N config : any user can be served by any backend server

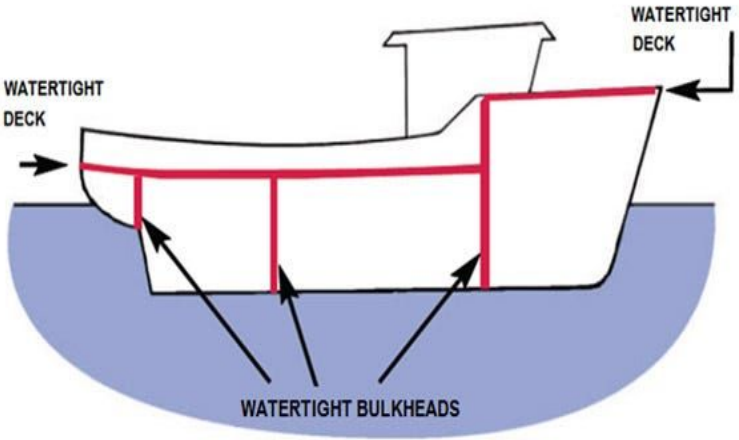
If something goes down, all users may get affected

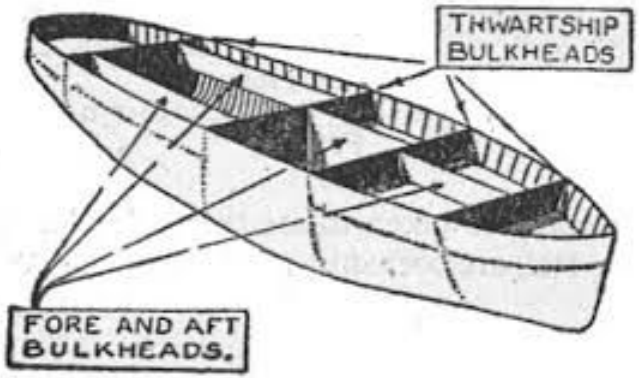
Difficult to figure out system limits in a large deployment of servers



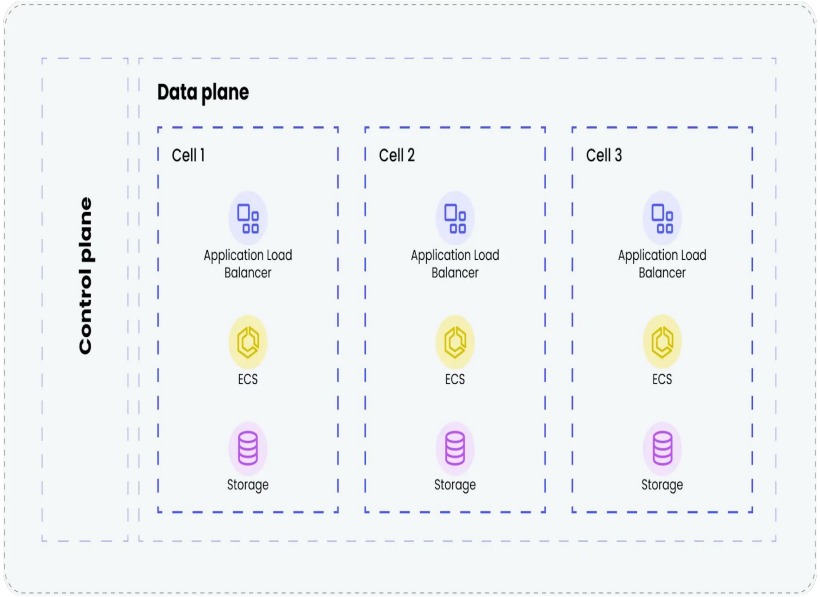
Ships have bulkheads

Water leaks in one compartment don't spread to another





Create cells for isolation



Cell = a certified unit of deployment

One cell is tested to handle (say) 10K users

More users ? Spawn more cells

https://maddevs.io/blog/cell-based-architecture-vs-microservices/

Add shuffle sharding

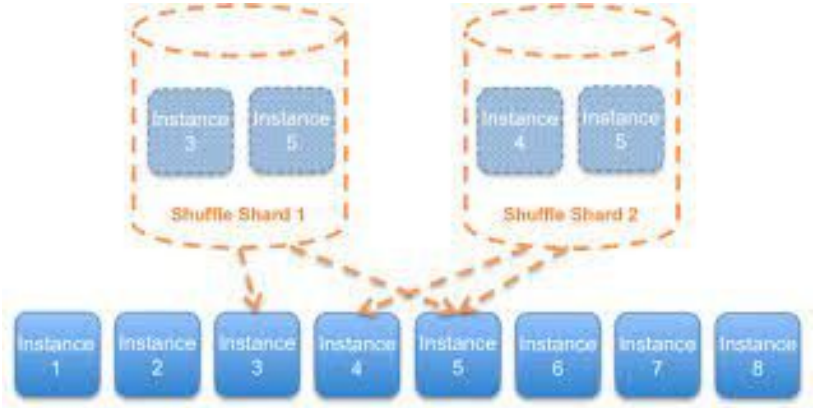
What if cell goes down - users of that cell affected

So user is connected to multiple cells for redundancy

But single user can only be connected to

specific predetermined set of cells

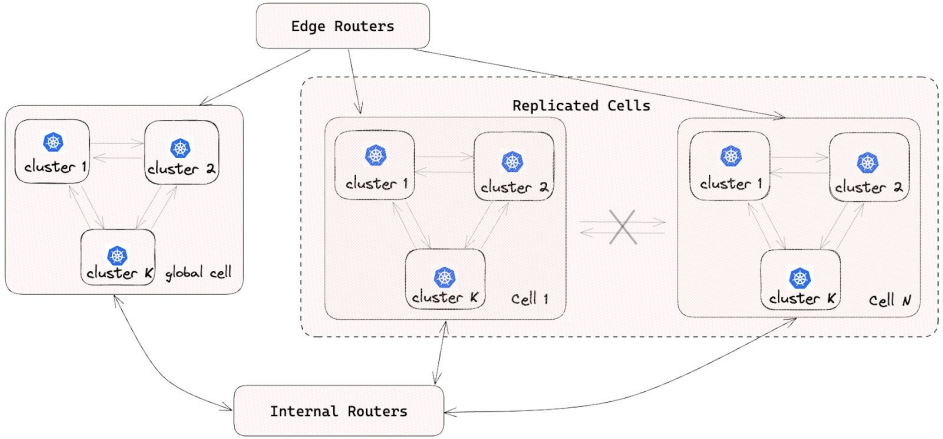
(e.g User 5 <=> 6, 12, 19)



https://aws.amazon.com/blogs/architecture/shuffle-sharding-massive-and-magical-fault-isolation/

Example : Doordash

All microservice pods are deployed in multiple isolated cells. Every service has one Kubernetes deployment per cell. To ensure isolation between cells, intercellular traffic is not permitted. This approach enables us to reduce the blast radius of a single-cell failure. For singleton services or those not migrated to the cell architecture, deployment occurs in a singular global cell.



https://aws.amazon.com/video/watch/520e6bc3923/

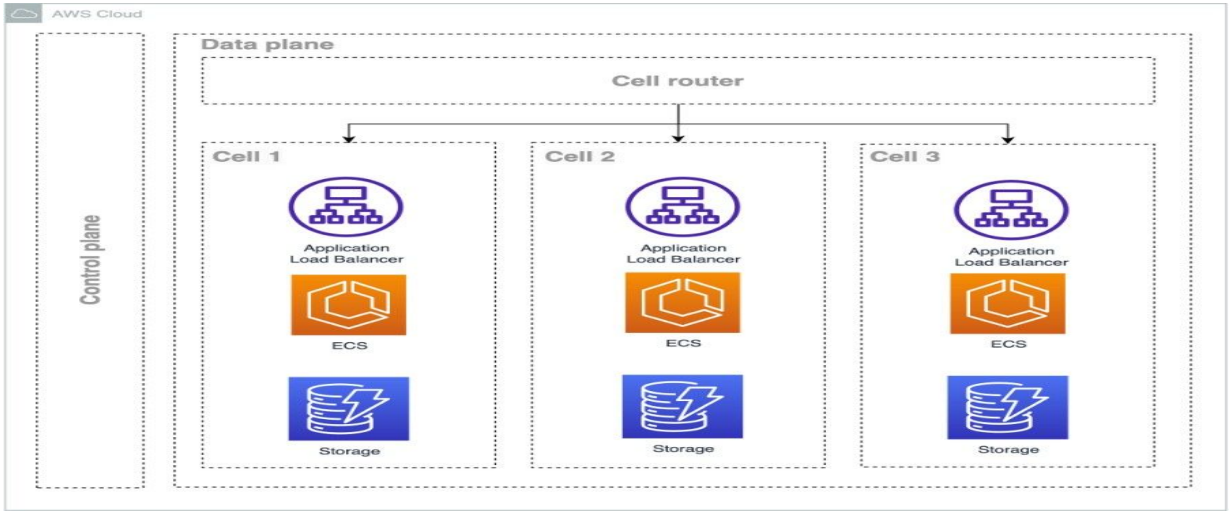
https://www.infoq.com/news/2024/01/doordash-service-mesh/

Example : Slack

....And so we finally arrive at our cellular architecture. All services are present in all AZs, but each service only communicates with services within its AZ. Failure of a system within one AZ is contained within that AZ, and we may dynamically route traffic away to avoid those failures simply by redirecting at the frontend.

https://slack.engineering/slacks-migration-to-a-cellular-architecture/

AWS guidance



https://d1.awsstatic.com/onedam/marketing-channels/website/aws/en_US/solutions/approved/documents/architecture-diagrams/a-cell-based-architecture-for-amazon-eks.pdf

https://docs.aws.amazon.com/wellarchitected/latest/reducing-scope-of-impact-with-cell-based-architecture/what-is-a-cell-based-architecture.html

Randomization

Correlated failure

You are running (say) 5 instances of a service

All instances of a Service keep getting same type of requests

They process them the same way

This increases probability of correlated failure

1. All run out of memory at same time (say due to garbage collection)
2. Their hash tables become identical; they hit the same bug and go down

Use jitter to reduce correlations

Randomize !

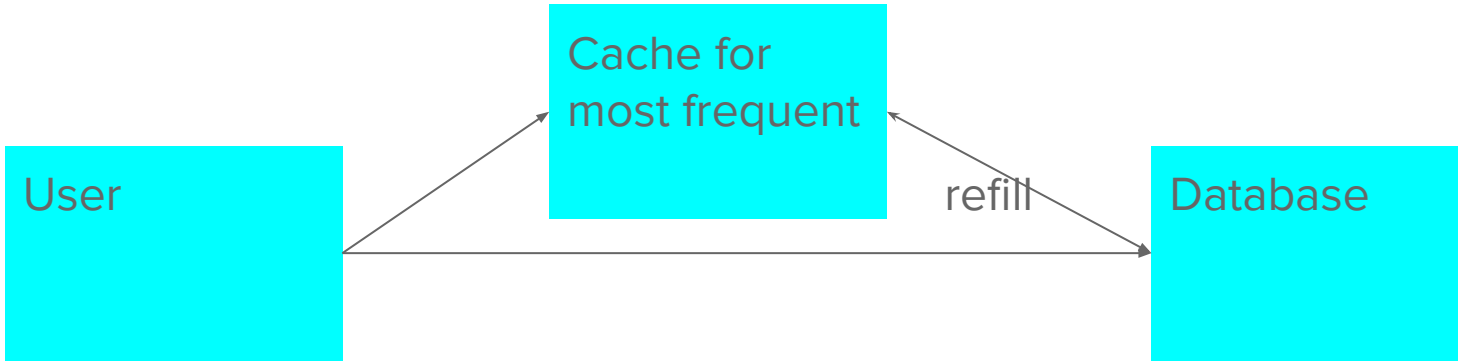
1. Memory heap size (N +/- 1 percent)
2. Queue size
3. Timeouts
4. Hash tables (add offset to the hash slot index)

Too much randomness can make debugging difficult

Therefore, use same random seed to drive all randomization

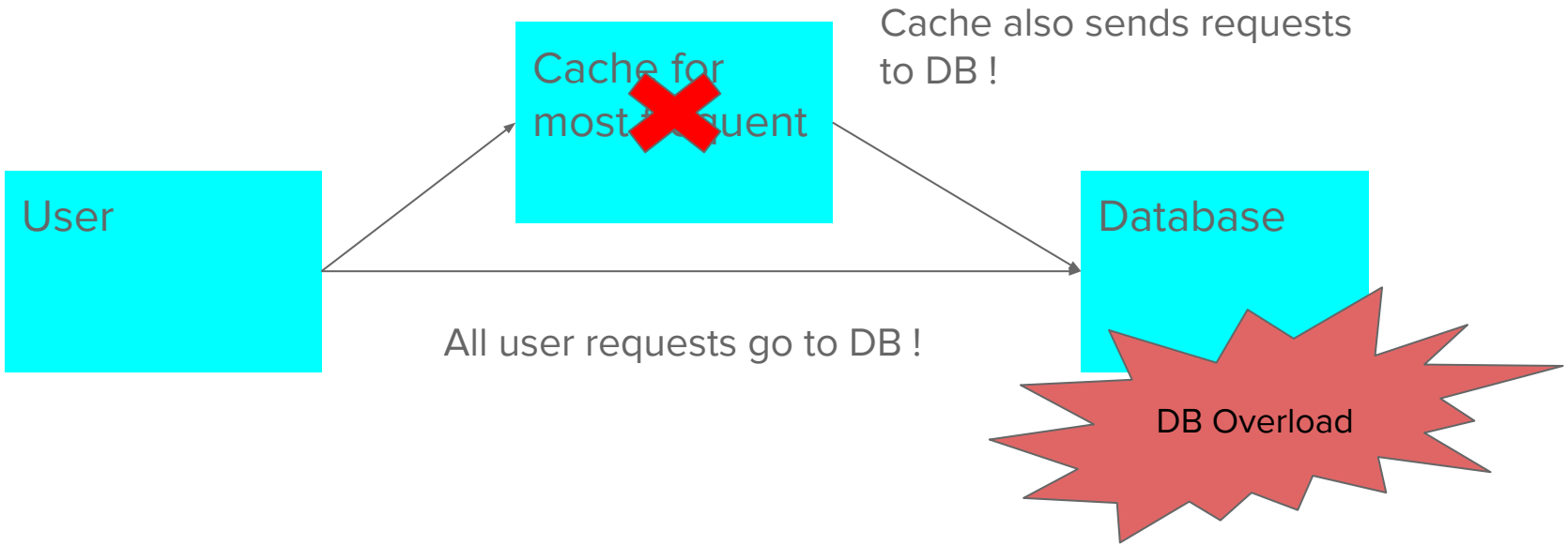
Caching

Naive caching



users see speed-up; everyone happy

When cache fails



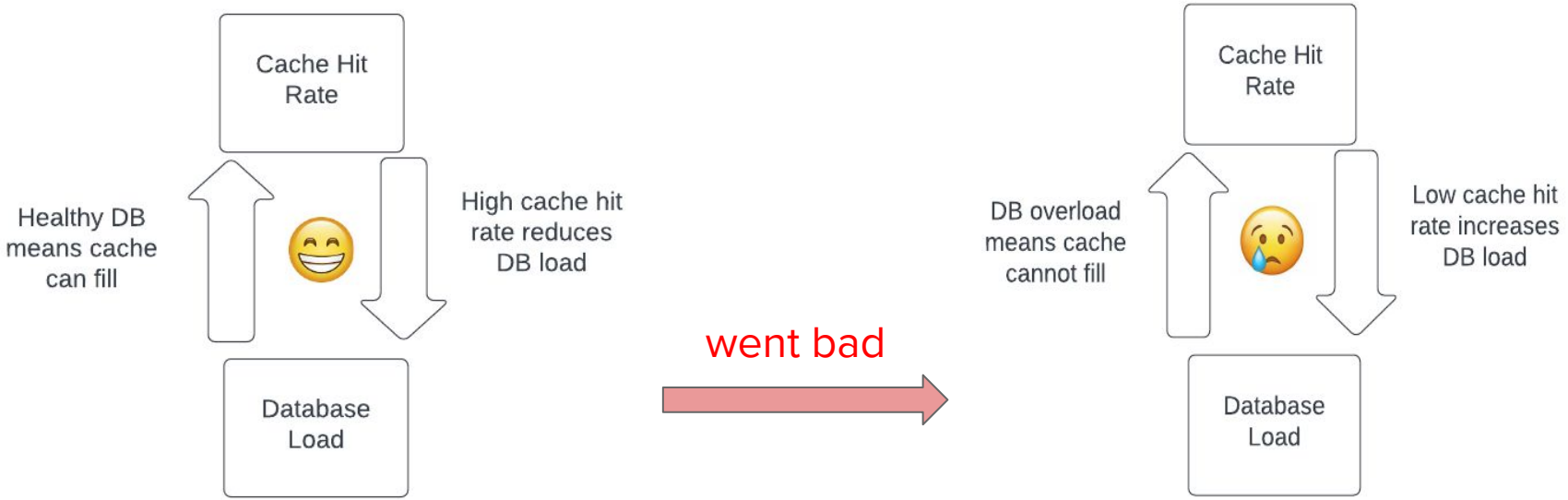
Example from Slack

https://slack.engineering/slacks-incident-on-2-22-22/

> *...With a significant portion of the cache unavailable, nearly all users needed to successfully query every shard of the keyspace. These queries are fast, but during this incident we simply had too many of them. The database was overwhelmed as the read load increased superlinearly in relation to the percentage of cache misses.*

Example from Slack

https://slack.engineering/slacks-incident-on-2-22-22/



How to prevent

What are your thoughts ?

How to prevent

Hot keys : request coalescing (other clients wait for first request)

How to prevent

Hot keys : request coalescing (other clients wait for first request)

Protect DB : must do rate-limiting, load-shedding

How to prevent

Hot keys : request coalescing (other clients wait for first request)

Protect DB : must do rate-limiting, load-shedding

Stagger cache expiry : Don't let all expire at same time - add jitter to cache TTL

How to prevent

Hot keys : request coalescing (other clients wait for first request)

Protect DB : must do rate-limiting, load-shedding

Stagger cache expiry : Don't let all expire at same time - add jitter to cache TTL

Proactive refresh : (refresh cache before expiry)

Don't be perfect : Return stale data or defaults or partial data

Anything else ?

Final thoughts

Read post-mortems

All good tech companies publish post-mortems of failures

Read them !

https://github.com/danluu/post-mortems

Metastable failures paper

Metastable Failures in Distributed Systems

Nathan Bronson*
Rockset, Inc.

Abutalib Aghayev
The Pennsylvania State University

Aleksey Charapko
University of New Hampshire

Timothy Zhu
The Pennsylvania State University

When large systems fail, they cannot recover

They remain stuck in "metastable" state

Due to "*work amplification*"

Individual components have recovery mechanism => but it causes overall collapse

recovery mechanism like failover, retries, autoscaling

https://sigops.org/s/conferences/hotos/2021/papers/hotos21-s11-bronson.pdf

Summary

Retries : make them adaptive (client <=> server)

Cells : to reduce blast radius

Randomize : to avoid correlated failures

Caching : adaptive to protect the DB (cache <=> DB)

fallback behaviour

control vs data plane

bound and limit everything - keep modes simple - fallback

https://github.com/sanjosh/csdocs/blob/master/distributed_systems/at_large_scale.md

https://github.com/sanjosh/csdocs/blob/master/distributed_systems/hints_for_design.md

https://github.com/sanjosh/csdocs/blob/master/distributed_systems/metastable_failures.md

https://github.com/sanjosh/csdocs/blob/master/distributed_systems/system_problems.md

https://github.com/sanjosh/csdocs/blob/master/general/architecting.md